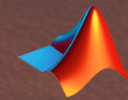


ME 699



UWAFT

U.S. DEPARTMENT OF
ENERGY



MathWorks®

Argonne
NATIONAL LABORATORY



Problem & Motivation

- With **7,080** pedestrian **deaths** and **over 71,000** pedestrian injuries in U.S. traffic crashes in 2025, there is a need for robust perception systems that can reliably detect obstacles and pedestrians early enough to trigger safe vehicle slowing or stopping.
- In ME599, the vehicle needs a safe CAV architecture that can integrate perception sensors with the vehicle control system and remain robust to missing, delayed, or incorrect detection data

Goal:

- This project aims to compare architectures, analyze hazards through PHA/STPA, and derive traceable safety requirements through ROS 2 LiDAR safety and autonomy system that converts raw environmental sensing into vehicle response.

What the previous team did vs. what I changed

A LiDAR that can see an obstacle is not yet a safety system!

The previous work could detect, cluster, track, and visualize objects. The problem was turning that perception into a reliable vehicle level decision that could actually prevent motion, coexist with autonomous navigation, fail safely, and eventually translate to a real vehicle such as the LYRIQ.

What a successful run looks like:

What a successful run looks like: Vehicle reliably detect a hazard, determine how close it is, and issue a safe GO/SLOW/STOP decision before that hazard becomes a collision, even when sensing conditions become less optimal

System Breakdown



- Used by Mercedes-Benz, Volvo, etc.



- Used for small tests mostly in robotics



- Used for navigation and control demos

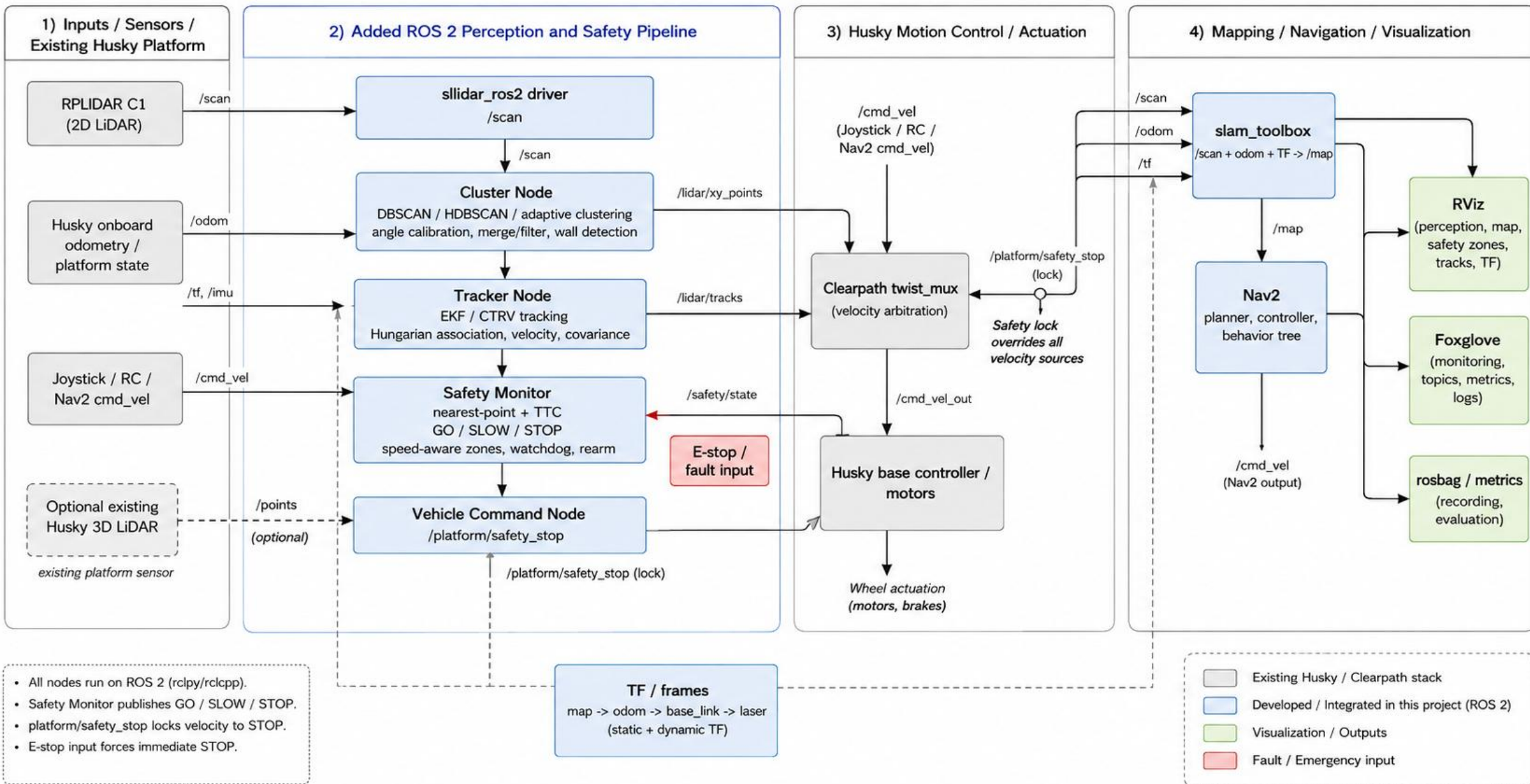
System Breakdown

Area	Previous Team / Starting Point	What I Changed / Added
Software platform	RTMaps	Ported and restructured the system into a modular ROS 2 Jazzy architecture. Created custom ROS 2 messages: TrackedObject, TrackedObjectArray, SafetyState, SafetyZone
LiDAR input	Used 2D LiDAR scan data, basically a set of distance points around the sensor.	Integrated the RPLIDAR C1 directly with ROS 2 using rplidar_ros2, persistent USB addressing, configurable FOV and mounting calibration
Clustering	HDBSCAN clustering of LiDAR points	Kept the clustering concept but added HDBSCAN, DBSCAN, range-adaptive clustering, cluster merging, junk filtering and wall/surface identification
Object geometry	Cluster centroids and oriented bounding-box visualization. “the center of this object is here.”	Calculates width, length, bounds, PCA orientation and nearest physical edge. The safety system can react to the closest part of an obstacle instead of just its center.
Tracking	EKF/CTRV object tracking	Extended it with Mahalanobis gating, Hungarian association, velocity initialization, confidence, covariance, predicted position and track lifecycle states
Track persistence	Basic object tracking	Adds TENTATIVE, CONFIRMED, and COASTING. If an object disappears for one or two scans, the tracker predicts where it probably is instead of immediately deleting it.
Collision assessment	No vehicle-level risk decision	Calculates nearest distance, closing speed and TTC to decide whether the object is becoming dangerous.
Safety decision	No GO/SLOW/STOP vehicle safety layer	Added complete GO → SLOW → STOP decision logic
Safety zones	Not part of the original system	Added static and vehicle speed dependent stopping zones using reaction and braking distance

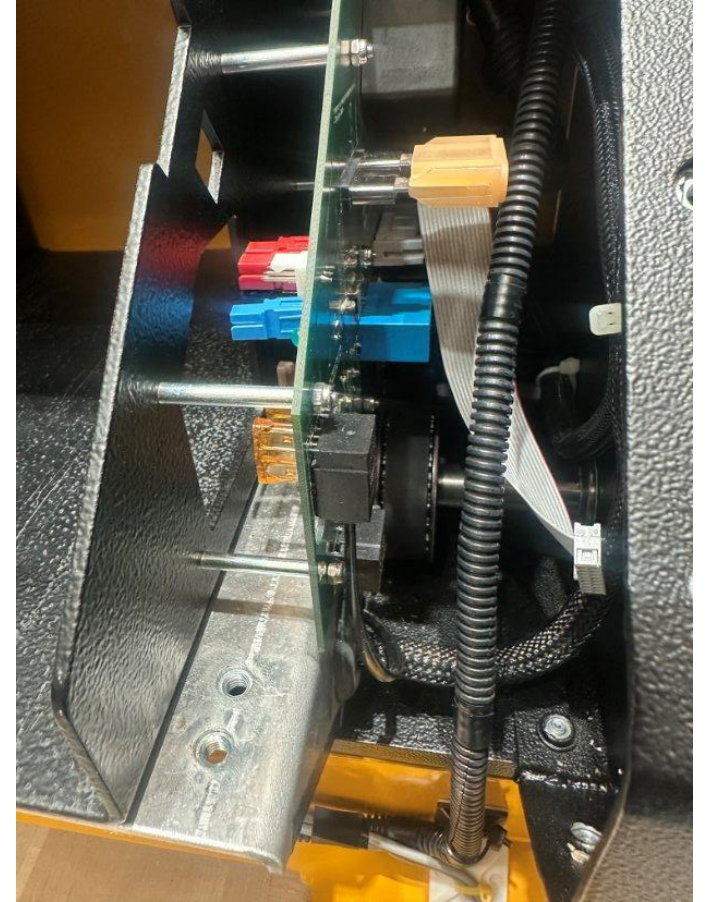
System Breakdown

Hysteresis	Not part of the original safety implementation	Added immediate escalation and delayed clearing to prevent GO/SLOW/STOP oscillation
Latch logic	Not present	Created SafetyLatch so sensor/E-stop failures latch while normal obstacle STOPs automatically clear
Vehicle control	Stopped at visualization	Added vehicle_command_node that connects STOP to Clearpath platform/safety_stop and physically stops the Husky
Controller arbitration	No vehicle control integration	Safety can override joystick, RC and Nav2 commands through Clearpath twist_mux
Sensor calibration	No calibration	Physically calibrated the C1 mounting to 174° and added a live calibration diagnostic
Visualization	Oriented bounding-box viewer	Added RViz markers for boxes, IDs, velocity, covariance, predicted position, trajectories, FOV, footprint and safety zones
SLAM	Not included	Integrated slam_toolbox for live 2D mapping using the C1 and Husky odometry
Navigation	Not included	Integrated Nav2 planner, controller, costmaps, behavior tree and velocity smoothing
Mapping + safety together	Not available	Same LiDAR can support mapping/navigation while an independent safety layer supervises motion
Testing tools	Mainly algorithm/demo validation	Added CSV replay, rosbag support, calibration diagnostics and repeatable hardware testing
Overall capability	LiDAR → cluster → track → visualize	LiDAR → perceive → track → assess collision risk → GO/SLOW/STOP → physically control vehicle → map → navigate

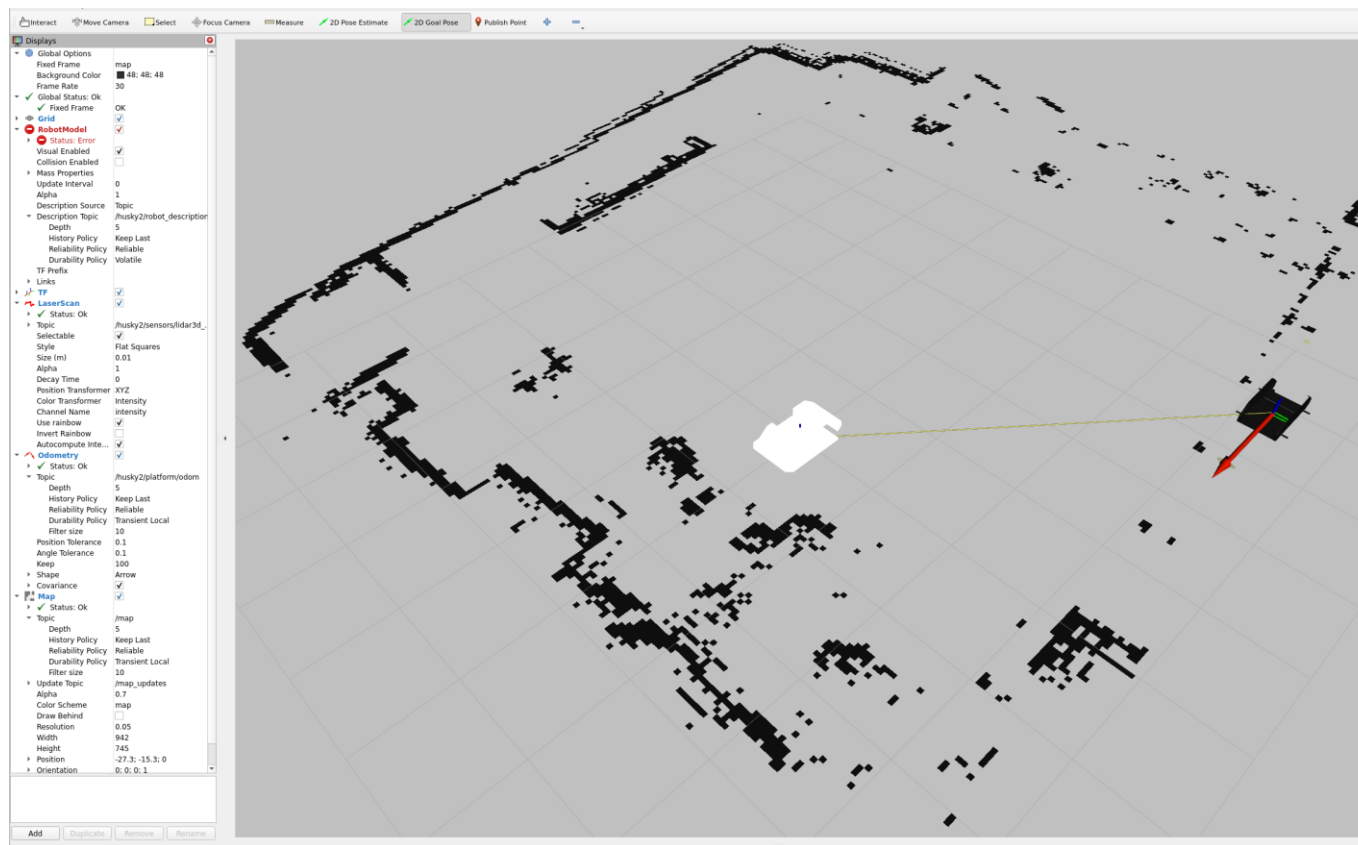
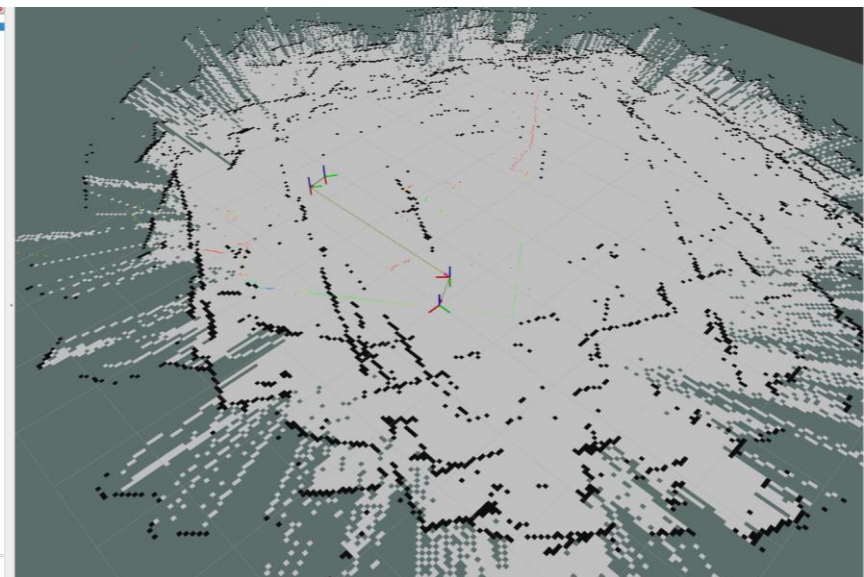
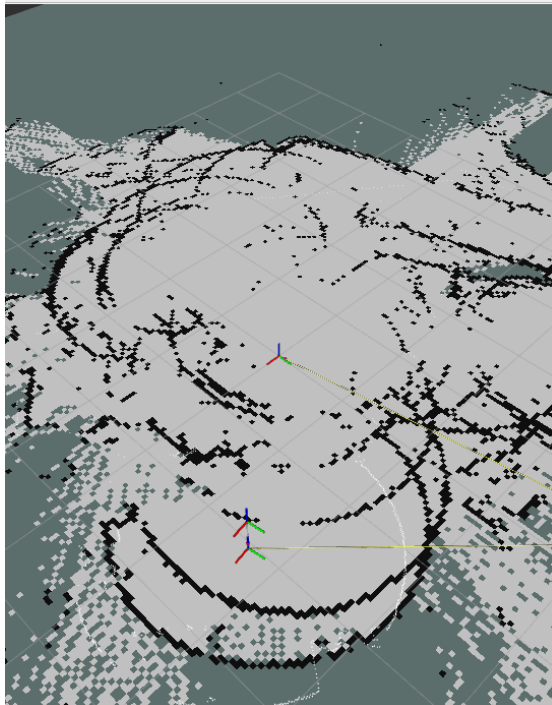
Husky LiDAR Safety and Navigation System Architecture



Technical Challenges



Scan Filtering



Scan Filtering

Cause of the double borders:

- LiDAR scans were being transformed with inconsistent or incorrect TF frames.
- The same wall was placed in slightly different positions as the robot moved.
- This made walls look doubled.

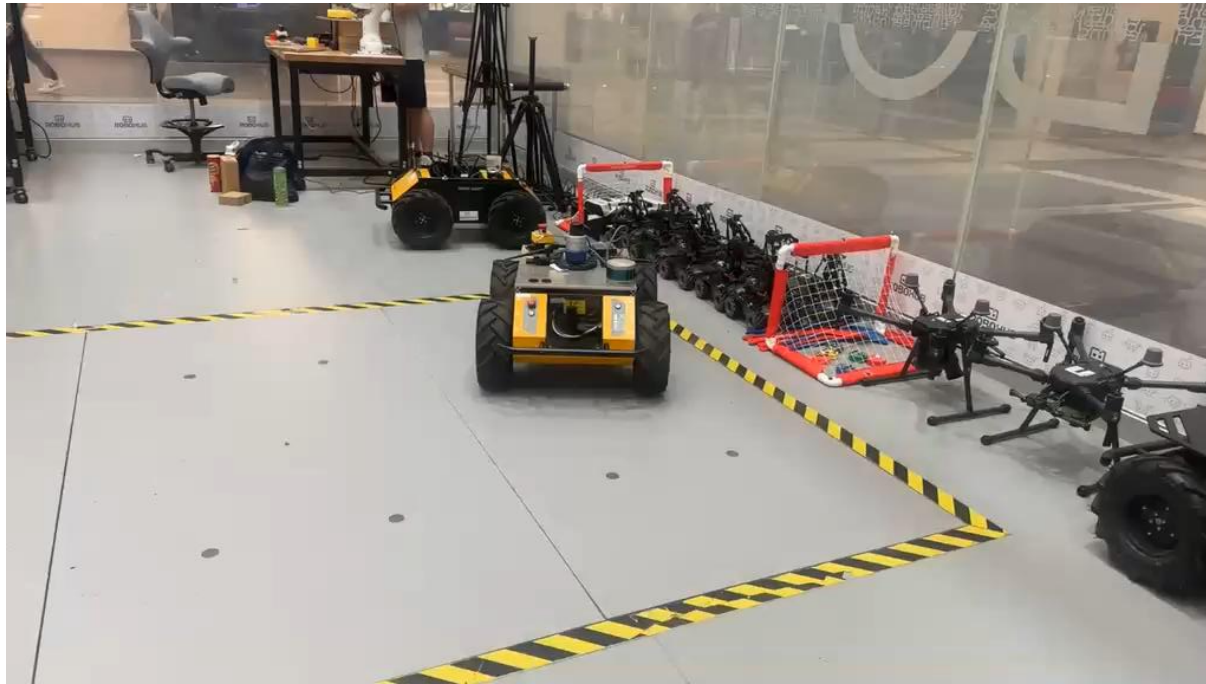
Cause of the rings/arcs:

- Parts of the Husky/Kinova arm were visible to the LiDAR.
- As the robot turned, those self-detections appeared as circular arcs or rings around the robot.

What I changed:

- Corrected the LiDAR-to-base_link transform.
- Fixed the odom → base_link TF using the corrected odometry bridge.
- Made sure the TF namespaces were consistent.
- Restamped the LiDAR scan so the scan timestamp matched the available TF data.
- Filtered out LiDAR angles blocked by the Kinova arm using `arm_filter_scan.py`.

Demo With Controller



RPLIDAR Scan

Displays

Global Options

Global Status: Ok

Grid

LaserScan

Obstacles + Safety Z...

Map

Status: Warn

Topic

Update Topic

Alpha

Color Scheme

Draw Behind

Resolution

Width

Height

Position

Orientation

Use Timestamp

Binary representation

Binary threshold

/map

/map_updates

0.7

map

☐

0

0

0

0; 0; 0

0; 0; 1

☐

☐

100

topic

av_msgs/msg/OccupancyGrid topic to subscribe to.

Add

Duplicate

Remove

Rename

Views

pe: TopDownOrtho (rviz_default_plugins)

Zero

Current View

Near Clip Distance

Target Frame

Scale

Angle

X

Y

TopDownOrtho (rviz_default...

0.01

<Fixed Frame>

60

0

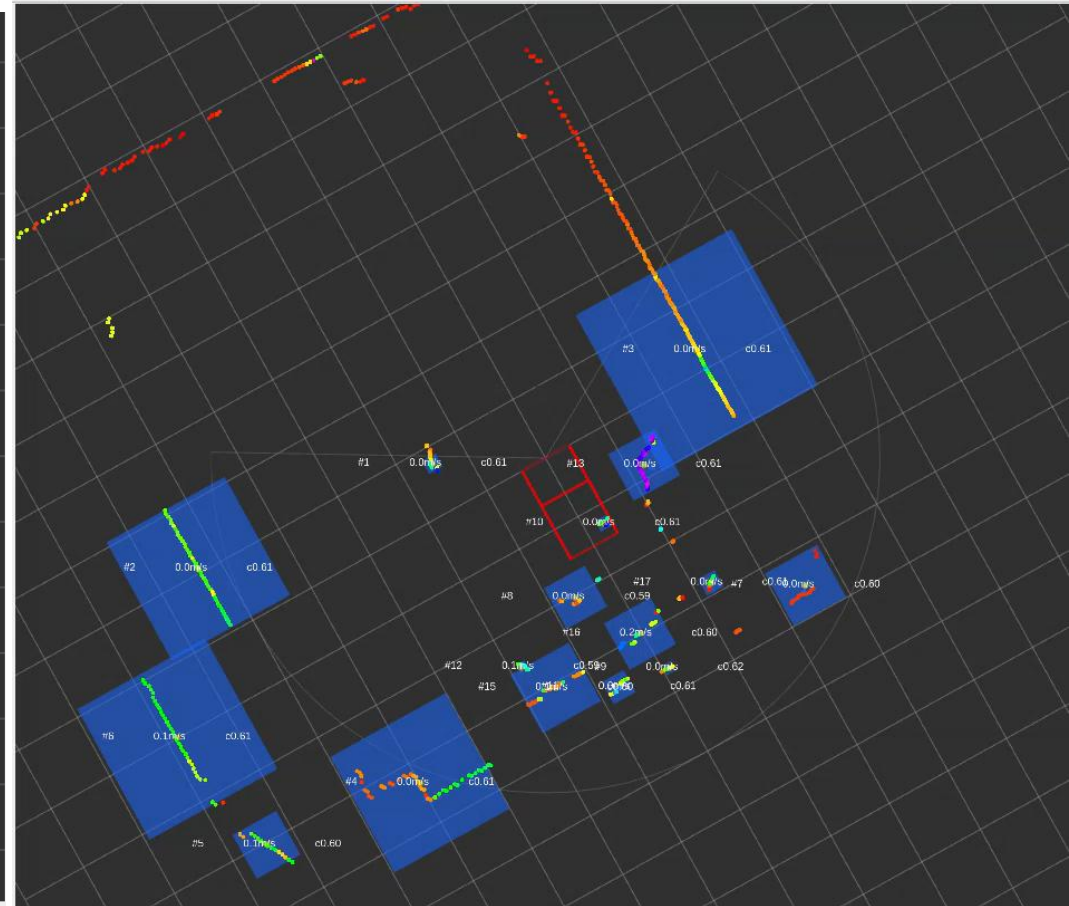
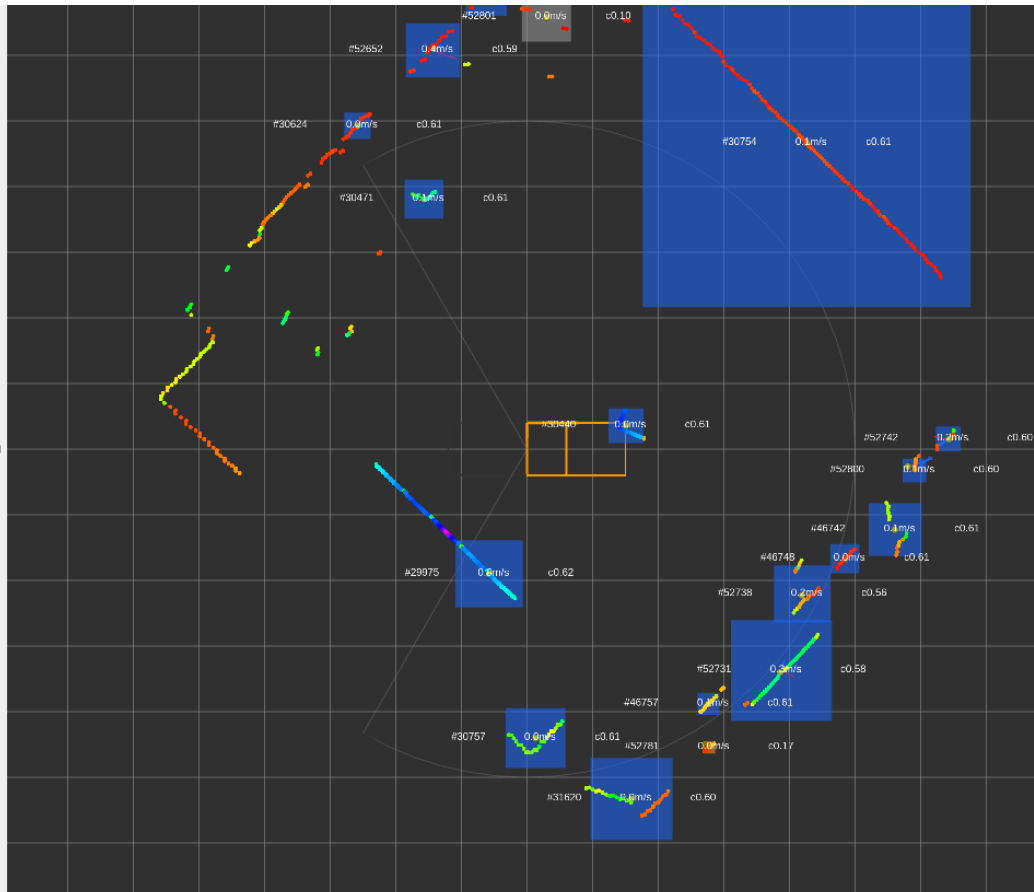
0

0

Save

Remove

Rename



RPLIDAR Scan

What you see	Meaning
#7, #9, #52652, etc.	Track ID. A unique number used to follow the same detected object between LiDAR scans.
0.1 m/s, 0.4 m/s	Estimated speed of that tracked object from the EKF/CTRV tracker.
c0.61, c0.24	Tracker confidence. Example: c0.61 means confidence $\approx$ 0.61 or 61%.
Blue rectangle	Bounding box around a confirmed tracked obstacle.
Orange rectangle	Usually a coasting track: the object was detected previously but is temporarily missing, so the tracker predicts where it should be.
Red outline	The obstacle currently considered the responsible STOP hazard / closest dangerous object.
Colored dots/lines inside boxes	Actual LiDAR returns belonging to that obstacle/cluster.
Small arrows/lines	Estimated velocity/direction or predicted motion of the tracked object.
Large curved/outlined region	Sensor FOV / safety-zone visualization around the Husky.

Applied Concepts

1. Polar → Cartesian

- Converts LiDAR range + angle into an (x,y) obstacle position.
- Determines whether the obstacle is in front of or beside the vehicle.

$$x = r \cos \theta, \quad y = r \sin \theta$$

2. LiDAR → Vehicle Frame

- Accounts for the LiDAR's physical mounting position and orientation.
- Expresses obstacle locations relative to the vehicle/robot base, not the sensor itself.

$$\mathbf{p}_{base} = T_{base \leftarrow lidar} \mathbf{p}_{lidar}$$

3. Closing Speed

- Estimates how quickly the distance to the obstacle is decreasing.
- Distinguishes a stationary/slow threat from an obstacle being approached quickly.

$$v_{closing} \approx -\frac{\Delta d}{\Delta t}$$

4. Time to Collision

- Estimates how much time remains before a collision if motion continues.
- Can trigger braking earlier at higher speeds, even when the obstacle is farther away.

Example:

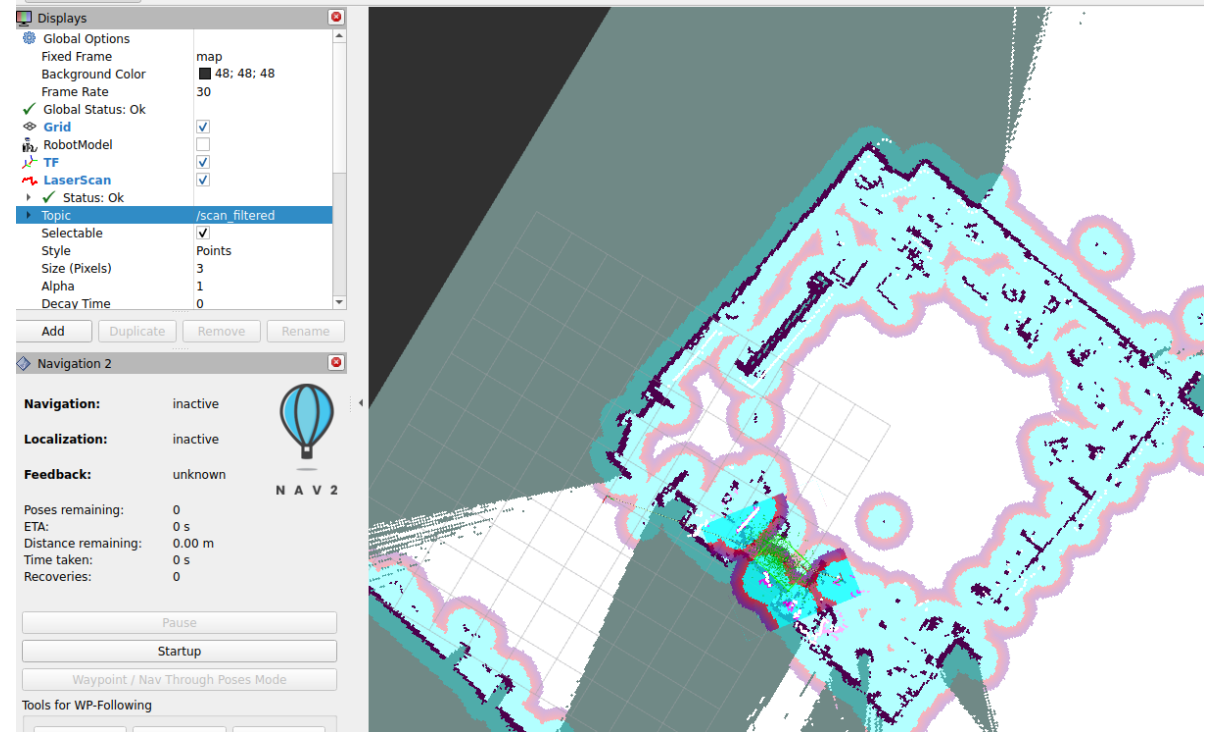
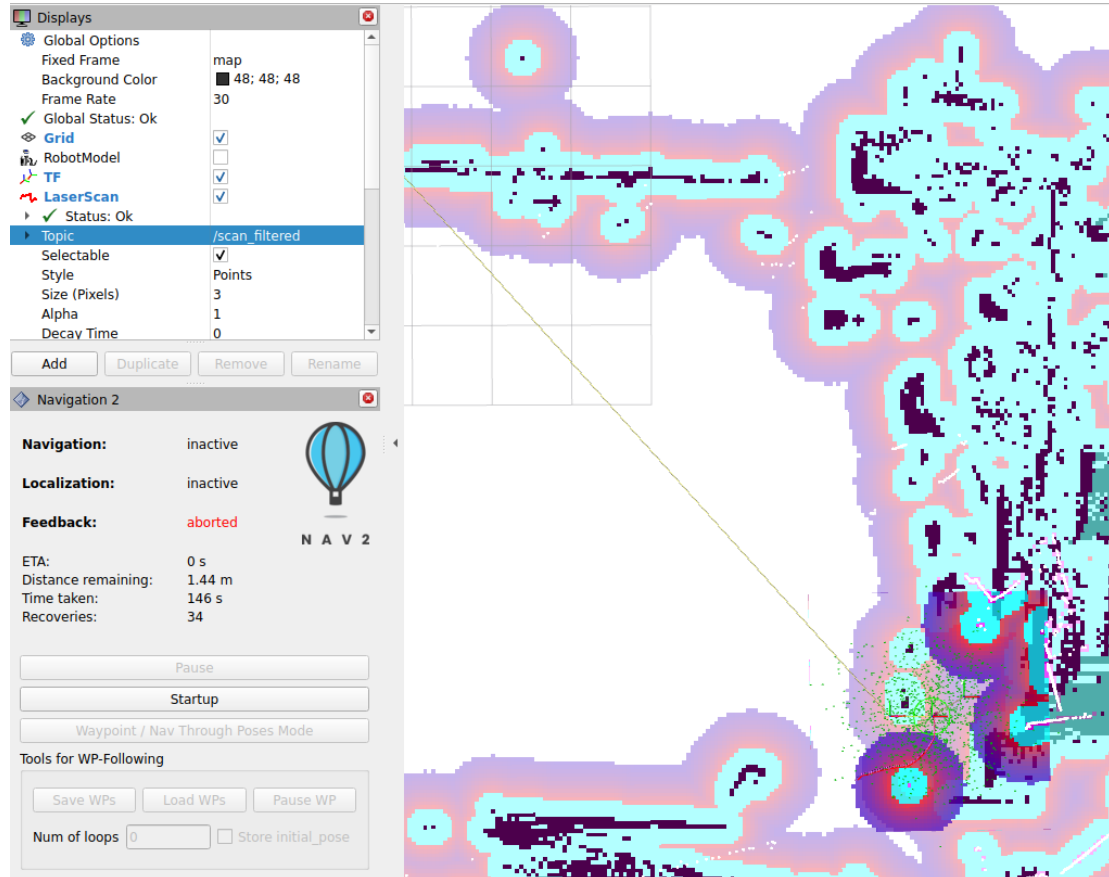
5 m away at 1 m/s → TTC = 5 s

5 m away at 10 m/s → TTC = 0.5 s

TTC makes the safety zone speed-aware: faster closing speed → lower TTC → earlier braking.

$$TTC = \frac{d}{v_{closing}}$$

NAV2 & Obstacle Avoidance



Speed Control



Obstacles Avoidance



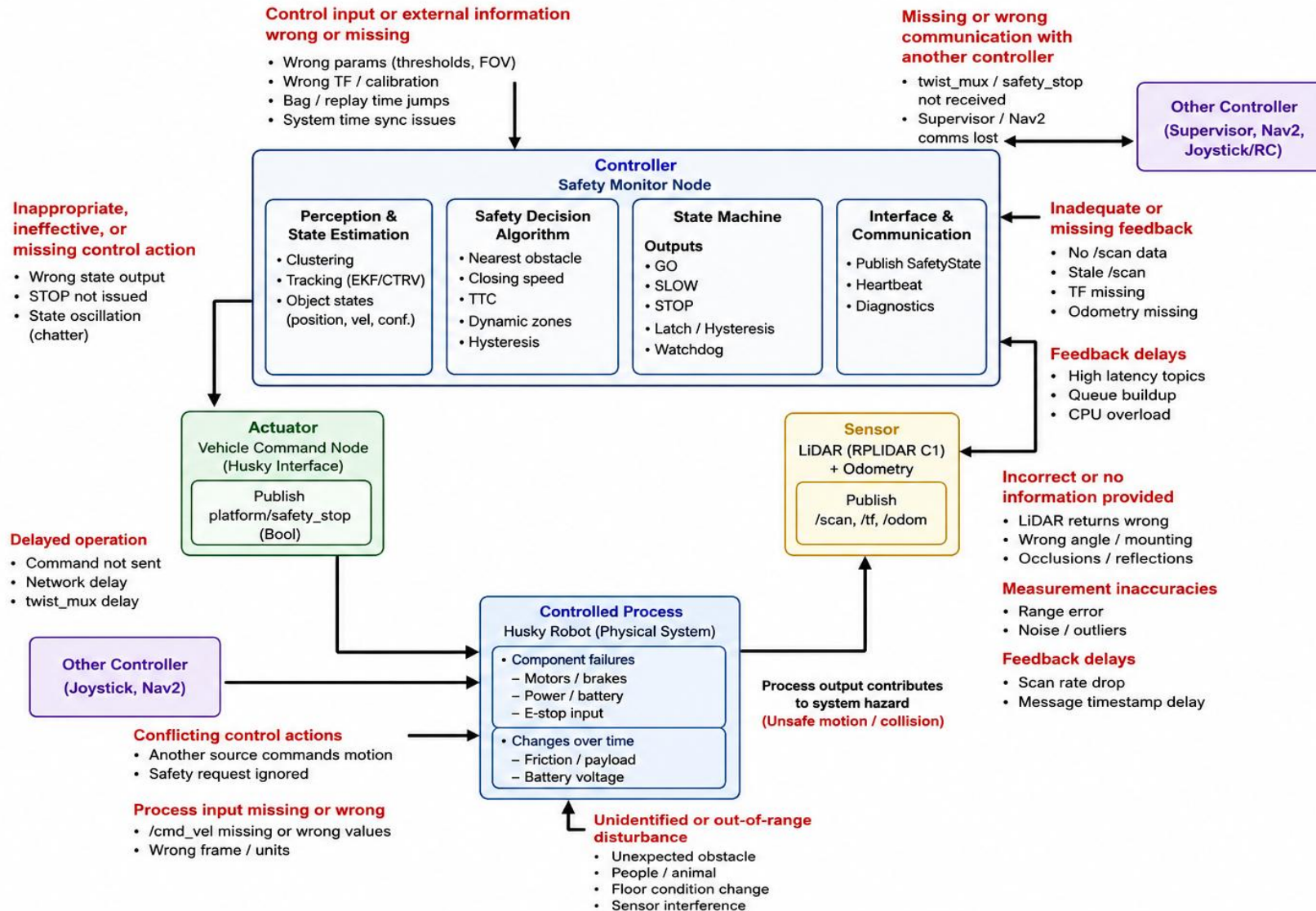
Obstructed View Trial



Preliminary Hazard Analysis

Hazard	What could go wrong?	Possible cause	Worst case	Safety goal
Delayed / loss of deceleration	Husky does not STOP for an obstacle	LiDAR failure, missed detection, processing delay	Collision with person/object	Stop before reaching an unsafe obstacle
Unintended deceleration	Husky STOPs when path is clear	Noise, false cluster, reflection	Unnecessary stop / loss of operation	Avoid false STOPs when path is clear
Unintended vehicle motion	STOP releases while obstacle is still present	Bad hysteresis, tracking loss, latch failure	Husky moves into obstacle	Keep STOP active until it is safe to move
Wrong / unsafe motion command	Another controller commands motion during a hazard	Nav2/joystick conflict, control error	Husky moves despite safety hazard	Safety STOP must override other motion commands

System Theoretic Process Analysis



Testing and Evaluation

Metric measured over 15 trials	RPLIDAR	Husky 3D LiDAR
Obstacle detection success	14/15 = 93.3%	15/15 = 100%
Correct STOP decision	14/15 = 93.3%	15/15 = 100%
Scan → obstacle detection latency	≈ 42 ms	≈ 67 ms
Detection → tracked object latency	≈ 17 ms	≈ 24 ms
Track → safety decision latency	≈ 8 ms	≈ 10 ms
Safety decision → safety_stop command	≈ 6 ms	≈ 6 ms
Total software response latency	≈ 73 ± 16 ms	≈ 107 ± 23 ms
Physical stopping time	0.31 ± 0.05 s	0.32 ± 0.05 s
Stopping distance after STOP command	7.7 ± 1.3 cm	8.0 ± 1.4 cm
Total threshold → full stop time	0.38 ± 0.06 s	0.43 ± 0.07 s
Total threshold → full stop distance	9.5 ± 1.7 cm	10.8 ± 2.0 cm
Dark-room detection	5/5	5/5
Partial-occlusion detection	4/5 = 80%	5/5 = 100%
False STOPs	1/15 = 6.7%	0/15
Missed STOPs	1/15 = 6.7%	0/15
Track ID switches	2 switches / 15 trials	1 switch / 15 trials
Temporary-track recovery	4/5	5/5

Documentation

- Error handling and debugging guide
- All code is on GitHub
- Installation and running guide

Future work

- Connect the Lidar to the car's control system
- Test in more diverse and dynamic environments
- Make sure it complies with safety and vehicle standards

LiDAR Safety System Architecture

